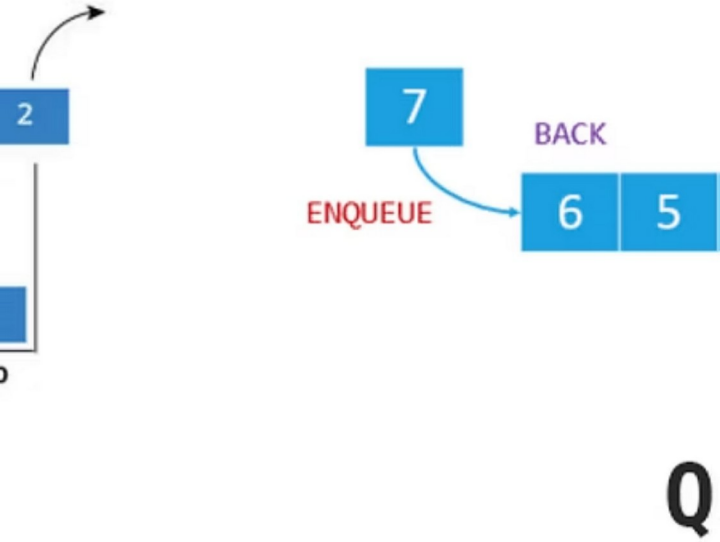
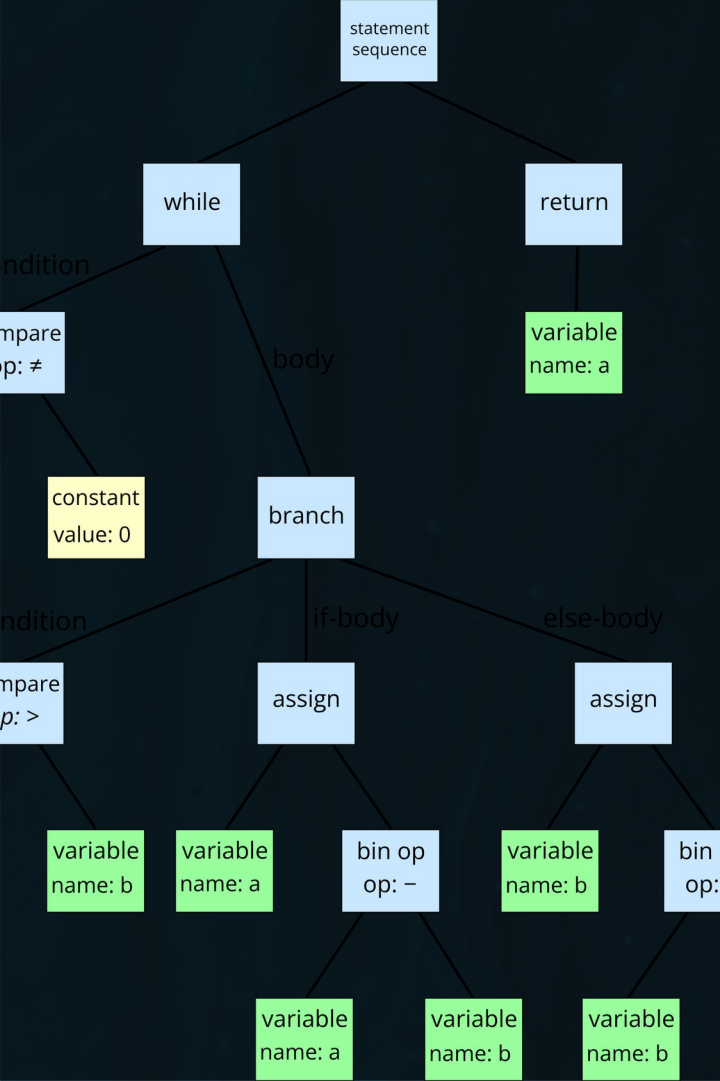


Structure



Autómatas de Pila

Un Autómata de Pila es una máquina teórica abstracta que extiende las capacidades de un autómata finito convencional mediante la adición de una **memoria auxiliar estructurada como pila**. Esta memoria de almacenamiento lineal ilimitado le permite recordar información arbitrariamente profunda, haciéndolo capaz de reconocer Gramáticas Libres de Contexto (GLC), las cuales se sitúan en el **Tipo 2 de la Jerarquía de Chomsky**.



Utilidades del Autómata de Pila

En ingeniería informática, los autómatas de pila son el **motor computacional del Análisis Sintáctico (Parser)** de un compilador. Las estructuras anidadas de los lenguajes modernos requieren mayor poder de cómputo que el que ofrecen las expresiones regulares y los autómatas finitos.

Validación Sintáctica

Análisis de la jerarquía gramatical de códigos fuente.

Evaluación de Expresiones

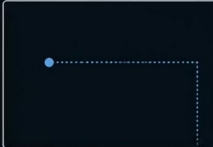
Procesamiento de fórmulas aritméticas y lógicas respetando la precedencia de operadores mediante estructuras de tipo árbol sintáctico.

Formatos Estructurados

Parseo de archivos JSON, XML o HTML basados en etiquetas y llaves de apertura y cierre balanceadas.

AFD/AFND vs. Autómatas de Pila

La diferencia fundamental radica en la **capacidad de memoria**: los autómatas finitos están limitados a sus estados, mientras que el AP dispone de una pila de tamaño ilimitado, lo que le otorga mayor poder de reconocimiento.

	AUTÓMATAS FINITOS (AFD/AFND)	AUTÓMATAS DE PILA (AP)
CAPACIDAD DE MEMORIA	estados finitos	pila infinita
TIPO DE LENGUAJE	Regular Tipo 3	Libre de contexto Tipo 2
EQUIVALENCIA DETERMINÍSTICA	AFD equivale a AFND	AP Determinístico es un subconjunto de AP No determinístico

La Limitación de los AFD y AFND

¿Por qué fallan los AF?

Los AFD y AFND carecen de capacidad para **contar o recordar eventos pasados** a gran escala. No pueden reconocer lenguajes simétricos como $a^n b^n$, ya que no tienen dónde almacenar el valor de n .

El poder de la pila

Al incorporar la pila, el AP «**escribe**» y «**saca**» **información** de la pila en lugar de almacenarla en sus estados. Los AP No Deterministas (APND) cubren la totalidad de las GLC, mientras que los AP Deterministas (APD) solo reconocen un subconjunto, suficiente para la sintaxis formal de la gran mayoría de lenguajes de programación.

cal 
ker.sock

 0  3 Total 6

althy

0.1%

2.4% (1.5 GB)

Stacks 3 3/0/0

Networks 6

 CPU & Mem

CPU

Memory

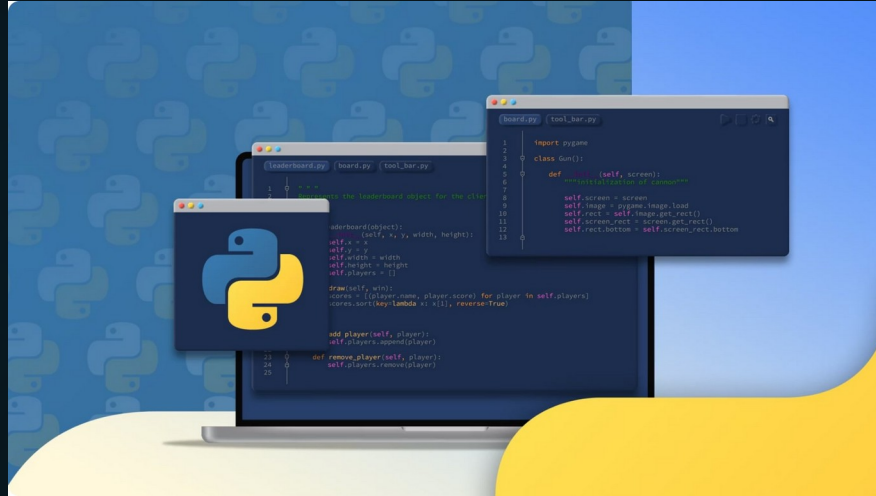
 Disk usage

Lexer para Archivos Docker

Analizador léxico diseñado e implementado desde cero en **Python** para la validación de archivos **Dockerfile**.

Fundamentado en la teoría de [autómatas finitos](#) y expresiones regulares para identificar todos los componentes léxicos del lenguaje.

Fase 1 — Definición de Tokens y Patrones



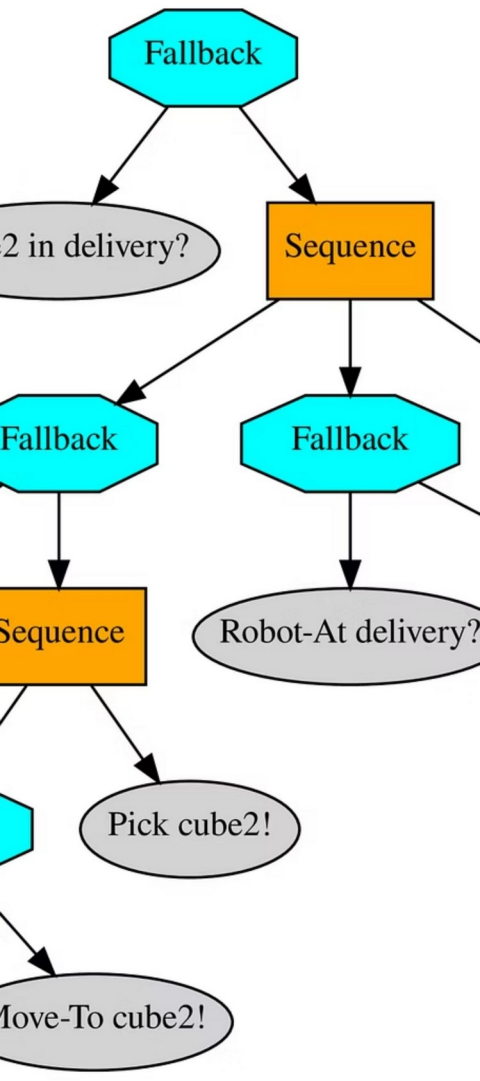
Instrucciones Docker identificadas

FROM · RUN · COPY · ENV · WORKDIR · EXPOSE
· CMD · ENTRYPOINT · VOLUME · USER · ARG

Se emplearon límites de palabra (\b) para evitar solapamientos con subcadenas.

Patrones adicionales

- **Comentarios:** iniciados por #
- **STRING:** cadenas entre comillas
- **Delimitadores:** ASSIGN, BACKSLASH
- **ARGUMENT:** argumentos y rutas generales



Fase 2 — Mapeo AFD y Compilación

Patrón maestro único

Todas las expresiones regulares se unifican en un único patrón con grupos nombrados: `(?P<nombre_token>patrón)`

Motor de Python

Procesado mediante `re.finditer`, simulando un **Autómata Finito Determinista (AFD)** que lee carácter por carácter.

Lectura secuencial

El flujo de entrada se recorre de forma eficiente en una sola pasada a lo largo del archivo.

Fase 3 — Control de Errores y Ubicación

Seguimiento de posición

El analizador rastrea activamente **línea y columna**. Cada salto de línea (`\n`) actualiza los contadores para calcular con precisión la coordenada de cada token.

Regla MISMATCH

Al final de la prioridad léxica se define una regla de salvaguarda. Si el puntero lee un carácter no contemplado, el lexer **interrumpe la ejecución** con un error detallado indicando la coordenada exacta del fallo.



Caso de Prueba 1 — Dockerfile estándar correcto

Archivo de entrada

```
# Imagen base de Python
FROM python:3.9-slim
WORKDIR /app
COPY . /app
RUN pip install -r
requirements.txt
```

Salida en consola

```
--- Analizando: Dockerfile_correcto ---
Token: COMMENT | # Imagen base de Python | L:1 C:0
Token: FROM | FROM | L:2 C:0
Token: ARGUMENT | python:3.9-slim | L:2 C:5
Token: WORKDIR | WORKDIR | L:3 C:0
Token: ARGUMENT | /app | L:3 C:8
Token: COPY | COPY | L:4 C:0
Token: RUN | RUN | L:5 C:0
```

Caso de Prueba 2 — Variables y puertos de red

Archivo de entrada

```
FROM alpine
ENV PORT=8080
EXPOSE 8080
```

Salida en consola

```
--- Analizando: Dockerfile_variables ---
Token: FROM | FROM | L:1 C:0
Token: ARGUMENT | alpine | L:1 C:5
Token: ENV | ENV | L:2 C:0
Token: ARGUMENT | PORT | L:2 C:4
Token: ASSIGN | = | L:2 C:8
Token: ARGUMENT | 8080 | L:2 C:9
Token: EXPOSE | EXPOSE | L:3 C:0
Token: ARGUMENT | 8080 | L:3 C:7
```

Caso de Prueba 3 — Detección de errores (MISMATCH)

Archivo con error

```
FROM ubuntu
RUN apt-get update %
```

📄 La regla **MISMATCH** captura el carácter `%` no contemplado y reporta la coordenada exacta del fallo léxico.

Salida en consola

```
--- Analizando: Dockerfile_error ---
Token: FROM | FROM | L:1 C:0
Token: ARGUMENT | ubuntu | L:1 C:5
Token: RUN | RUN | L:2 C:0
Token: ARGUMENT | apt-get | L:2 C:4
Token: ARGUMENT | update | L:2 C:12

[ERROR DETECTADO]
Error léxico: Carácter inesperado '%'
en la línea 2, columna 19
```



LENGUAJE Y COMPILADORES · TEMA 4

Lexer para un subconjunto de Rust

MiniRust: diseño del lenguaje L, construcción del analizador léxico con Flex, y verificación mediante casos de prueba.

¿Por qué Flex?

Metacompilador

Flex (Fast Lexical Analyzer Generator) recibe una especificación (archivo .l) con expresiones regulares por cada token, y genera automáticamente el código en C del analizador léxico — construyendo internamente el AFD correspondiente.

En vez de codificar el autómata a mano...

Flex traduce las expresiones regulares del lenguaje L directamente a un programa en C ejecutable.



El lenguaje L — “MiniRust”

Un subconjunto de Rust acotado, pensado exclusivamente para el análisis léxico

Declaración

```
fn · let · mut · struct · enum ·  
impl · const · static
```

Control de flujo

```
if · else · while · for · in ·  
loop · break · continue · return  
· match
```

Módulos y visibilidad

```
pub · use · mod
```

Tipos primitivos

```
i8...i64 · u8...u64 · f32 · f64 ·  
bool · char · str
```

Tipos de biblioteca

```
String · Vec
```

Literales

```
enteros · flotantes · booleanos  
· char · string
```

Literales, operadores y una regla clave

EXPRESIONES REGULARES DE LOS LITERALES

Entero `[0-9] ([0-9] | _)*` `1_000`

Flotante `[0-9]+\.[0-9]+` `3.14`

String `" ([^"\\n] | \\.) *"` `"MiniRust"`

Carácter `' ([^'\\n] | \\.) '` `'A'`



Comentarios (no generan token)

```
// comentario de línea  
/* comentario de bloque */
```

OPERADORES: COMPUESTOS ANTES QUE SIMPLES

Compuestos

`== != <= >= && ||`
`-> => :: += -= *= /=`

Simples

`+ - * / % = < > ! & | ^`



Flex aplica "coincidencia más larga". Si la regla de "=" apareciera antes que la de "==", igual se reconocería "=" correctamente, pero el orden explícito evita ambigüedades y refleja el criterio de desempate: ante longitudes iguales, gana la regla que aparece primero en el archivo .l.

Estructura de mini_rust.l

Todo archivo Flex se organiza en tres secciones separadas por %%

1

Definiciones

#include, macros de regex
reutilizables
(IDENT, INTEGER, FLOAT...),
%%option

```
IDENT [a-zA-Z_][a-zA-Z0-9_]*
```

2

Reglas

Patrón → acción en C.
Orden: comentarios → keywords
→ tipos →
literales → operadores →
puntuación → IDENT

```
"fn" { emitir("KW_FN",  
yytext); }
```

3

Código de usuario

Función emitir() y main(): abre el
archivo
de entrada y llama a yylex()

```
int main(int argc, char **argv)  
{...}
```


Tres casos de prueba

Cada archivo .rs ejercita una parte distinta de la especificación léxica

01



`ejemplo1_suma.rs`

Funciones, parámetros tipados, let mut, operador `->` y `+=`, return.

```
(KW_FN, 'fn')
(IDENT, 'suma')
(OP_ARROW, '->')
... 67 tokens · 0 errores
```

02



`ejemplo2_struct.rs`

struct, impl, self, if/else, match con `=>`, literales flotantes, char, bool y string.

```
(KW_STRUCT, 'struct')
(KW_IMPL, 'impl')
(OP_FATARROW, '=>')
... tokens · 0 errores
```

03



`ejemplo3_bucles_error.rs`

while / for / loop, operador `::` (Vec::new), y un error léxico intencional: `v@lor`.

```
ERROR LEXICO: '@' linea 28
... 95 tokens · 1 error
```

Control de errores léxicos

El analizador no se detiene en el primer error: lo reporta y continúa

ENTRADA

```
let mut v@lor = 10;  
contar(numeros);
```

regla genérica de error en mini_rust.l:

```
. { fprintf(stderr, "ERROR LEXICO..."); }
```



SALIDA (stderr + resumen)

```
ERROR LEXICO: caracter no  
reconocido '@' en la línea 28
```

--- Resumen ---

Tokens reconocidos : 95

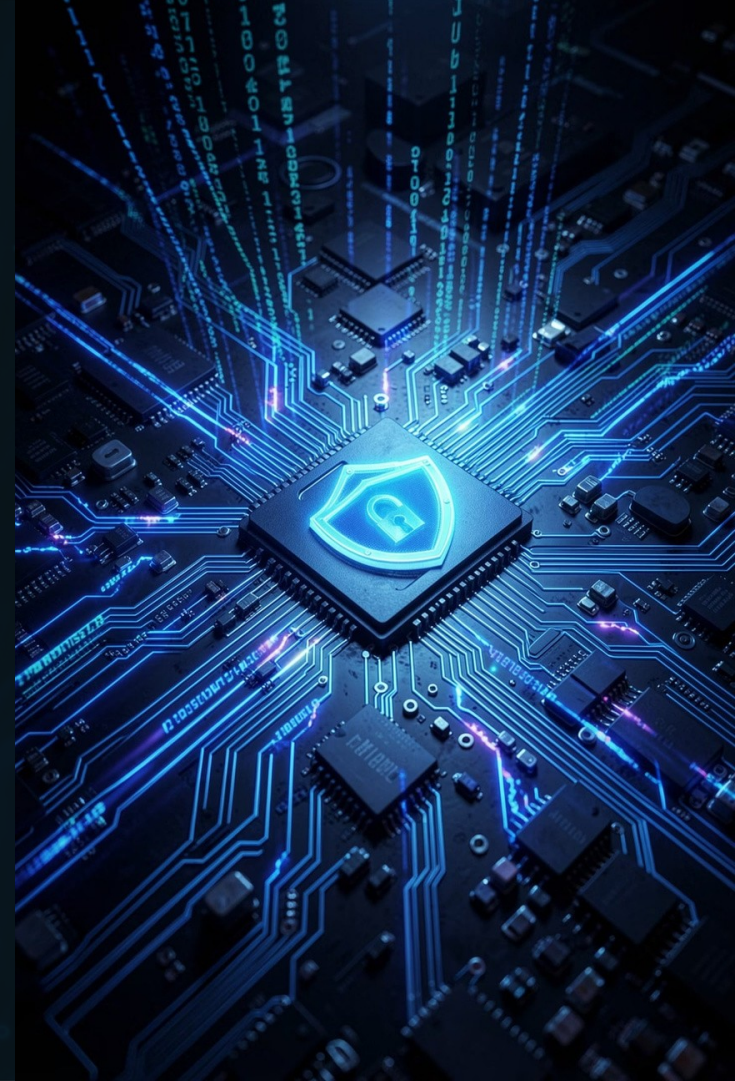
Errores lexicos : 1



Si el analizador se detuviera en el primer error, un compilador solo mostraría un fallo por ejecución. Al usar una regla de “estado fallido” que vuelve al estado inicial, MiniRust puede reportar varios errores léxicos en una sola pasada.

Metacompiladores en la Seguridad Informática

FLEX, Snort y YARA — motores invisibles de la ciberseguridad





La velocidad es una necesidad absoluta

El problema

IDS, SIEM y motores de malware procesan millones de registros en tiempo real. La latencia puede significar una brecha.

La solución: FLEX

Metacompilador que genera código C optimizado. Actualiza reglas sin reescribir el motor.

The image shows a computer monitor in a server room, displaying the NetFlow traffic analysis tool. The screen is divided into several sections:

- Top Bar:** Displays the application name "NetFlow" and a status bar with "39" and "Source: 10.0.0.1".
- Left Panel:** A sidebar with icons for various functions: "Host", "Packet Size", "Packet Count", "Packet Length", "Packet Protocol", "Packet Source", "Packet Destination", "Packet Time", "Packet Size", "Packet Count", "Packet Length", "Packet Protocol", "Packet Source", "Packet Destination", "Packet Time".
- Main Area:**
 - Packet Size Histogram:** A bar chart showing the distribution of packet sizes. The x-axis is labeled "Packet Size (bytes)" and the y-axis is labeled "Count".
 - Packet Size Line Graph:** A line graph showing the distribution of packet sizes over time. The x-axis is labeled "Time" and the y-axis is labeled "Count".
 - Packet Size Table:** A table with columns "Count", "Packet Size", "Packet Count", "Packet Length", "Packet Protocol", "Packet Source", "Packet Destination", "Packet Time".
- Bottom Right Panel:** A section titled "Alerts on Inventory" with a warning icon. It contains the text: "NetFlow traffic: 10.0.0.1 - 10.0.0.1", "NetFlow traffic: 10.0.0.1 - 10.0.0.1", and "NetFlow traffic: 10.0.0.1 - 10.0.0.1".

```
alert tcp any any -> 192.168.1.0/24
80
```

Palabras reservadas: alert, drop

Protocolos: tcp, udp

Direcciones, subredes, any

De la teoría a la práctica: YARA

Archivo pureba.yar — regla `Deteccion_Malware`

Sección strings

`$cadena1` → "cmd.exe /c"

`$hex_firma` → { E2 34 A1 C8 23 FB }

Sección condition

Regla activa si encuentra `$cadena1` **or** `$hex_firma`

El lexer: yara_lex.l



Desde las librerías hasta yylex(), cada patrón se traduce en un token listo para procesar.

Ejecución: de texto crudo a estructura de datos

01

TOKEN: RULE

Lexema: rule

02

TOKEN: IDENTIFICADOR

Lexema: Deteccion_Malware

03

TOKEN: STRINGS_SEC

Lexema: strings:

04

TOKEN: OP_OR

Lexema: or

```
1 //Echess Mokevnoniafles
2 1 Mvige-/(IQMLIORUNE = ctnkottane(""));
3 cahuramcces/(lkkeion = "revengy(");
4 /ökocteane./Lotraateivcfnckilcú(");
5 chzcrey#f./bolentluacFt= dc ,pípycbert.yia"}"eam(") af"nw");
6 nx/Éeoteccerlñac00okictoytanñ anokacewem'izconacLIENNERNV(");
7 cspcmis/fccotevtil:/"; 00MLU000VE-ctochan="xtcvane#";
8 an(/CQ/"V(RIGANI ; 0ctÉicncatip:"derflurcillf ((26");
9 vayinrtonecstimahTC" /, QowbQwroteVt"/Cemantteoiyt ();
10 zeciaE/"onntoiGkL/lango/aum ohvammtecktac/toaktweannol(/Caketok);
11 f//akcalete/amtod/-areyleneno//LeeRR" éiy("cgneg);
12 //ortolertectpohl/tecaRtoM_gtaxewhi';
13 V/m/totectertaV/ stoñansBraicoC/QU000/"ichuryce//a/enetnb");
14 V//okeactlnrnac"finmoh"-exbezceimio));
15 { ""qomtenlyst"o"pwni"ne "" strcvrmedacafapar_"wurclame"leuid#); }
16 cofccEmiBcortlehUma*Laod,bumcu.
17 comolinnactaebL0LlrnmEE0DOKAVIORUNE=/ERL5));
18 nmolegrtaking = foarkttat0Be=GLcermoucslr'sco0oursl"}
19 vn/ofsnctrnotickE =fh6e==_porylrevstav("nevasat#");
20 rnw/omctasrkirol/csrnbuñahicewaha"};
21 vn/raactertueelracioap/cunealsundotel$"?);
22 f//omnnstrthexcoktoexabkg, Kaopnm, corsialtow-xo/liopac/Gadt\ _shlecee(anos"Lihgy?(t""");
23 wyk6/O-LD ta6t/co6hmibND =8F0-" (fxkoc(Exuüdtoga));
24 r/ /orertertvanfostexhll/alcscststt}/...
25 shon/t/2P R=EIL30h;certam /0/taxotnmarvasferl/CebdlehCs]}
26 w/w/ohatrtoseñce/tonNDEF" can/t'pailen()/tocsteer"s(" ;
27 } inrceomely(aftc#alon( 1p))
```




Conclusión

Snort

Procesa paquetes de red
en tiempo real

YARA

Analiza binarios
maliciosos con precisión

FLEX

Transforma reglas en código C ultrarrápido



Los metacompiladores permiten a los sistemas de defensa **reaccionar a la misma velocidad** que los ataques.